
Hawkish-Dovish Index

Hsiu-Hsuan Yeh

Economics, National Taiwan University, Taipei, Taiwan

Abstract

This paper asks which design choices in a hawkish-dovish sentence classifier actually move its performance. Holding the labelled FOMC data of [1] fixed, we vary one component at a time: the strategy that pools token embeddings into a sentence embedding, the pooling layer BERT inherits from next sentence prediction, the encoder layers the embeddings are read from, the capacity of the encoder, and the corpus the encoder was pre-trained or fine-tuned on. Three findings come out of the sweep. RNN pooling trains the largest classification head of the three pooling strategies and still loses to a linear head on mean-pooled embeddings. The inherited pooling layer hurts under both mean and [CLS] pooling, and mean and [CLS] trade places from one encoder to the next, so neither of the two is a safe default. What separates models is instead what they were adapted to: the best dovish and the best hawkish scores both come from adapted checkpoints, one tuned on sentiment and one on sentence similarity, and no generic encoder of any size reaches either, although a plain roberta-base still holds the neutral class. We then define the hawkish-dovish index that aggregates these sentence labels, which is the object the classifier exists to serve.

1 Introduction

FED’s dual policy objective is to stabilize the price while keeping employment at its maximum level. FED has many tools and forward guidance has become an indispensable one. The motivation of forward guidance is to let the general public form the correct belief on whether FED will develop a contractionary (hawkish stance) or expansionary policy (dovish stance). Forward Guidance is implemented through various communicating channels. General public will take actions based on the belief and thereby the market is intervened and evolves toward the direction aligned with FED’s objective.

The rest of the paper proceeds as follows. Section 2 describes the data, Section 3 lays out the text classifier and the design choices we test, Section 4 defines the index built on top of the classifier, and Section 5 reports the classification results.

2 Data

We utilized the data published by [1] to build the hawkish-dovish text classifier as well as to build the hawkish-dovish indicator. For more information about how they construct the dataset and about several descriptive statistics regarding this dataset, please refer to their paper.

3 Classifier

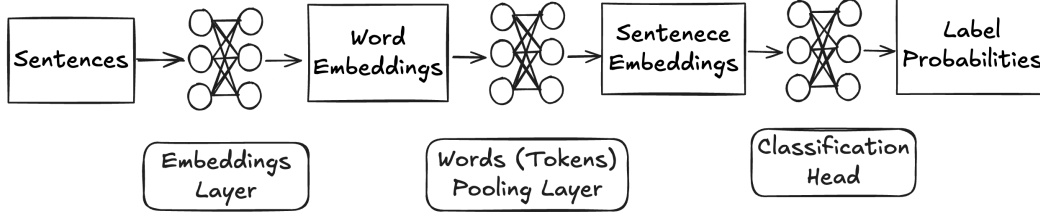


Figure 1: Text classification pipeline.

We design a high-level pipeline for text classification, demonstrated in Figure 1. Note that language models dominate this strand of text classification literature and we particularly put our focus on the so-called masked language models (MLMs), rather than the autoregressive language models, mostly known as large language models (LLMs), which induce the current AI hype. The most renowned MLMs is proposed by [2], named as BERT which is the foremost component to build our text classifier.

A good classifier should have capabilities to capture the sentence level semantics, and our task, which is to justify the sentence to have either hawkish, dovish or neutral attitude, therefore, we conjecture that a more domain specific pre-training might boost the performance. Consequently, we have similar experimental design but leverage other MLMs, pre-trained on more domain specific corpora. Beyond the standard supervised approach, meta-learning and semi-supervised training would let the classifier compile information beyond the semantics of the single sentence in front of it. We flag both as the natural extensions of this design and leave them to future work: every result below is supervised.

3.1 Vanilla BERT Classifier

Speaking to train a text classifier based on the pre-trained MLMs, it's intuitive to stack a linear classification head on top of a MLM and the role of MLM is to embed words to vectors contextually. For this type of architecture, an important decision is whether to update the weights of MLMs during the stage of backward propagation. Fine-tuning refers to the scenario that we update MLMs' weights during backward propagation and it's generally more computationally expensive and time consuming. In our text classification task, we freeze the weights of MLMs and merely extract word embeddings from them, which is what keeps the comparison across the many pre-trained models below affordable. Fine-tuning is left to future work, so every number we report should be read as the quality of an off-the-shelf representation rather than as the ceiling of a given model.

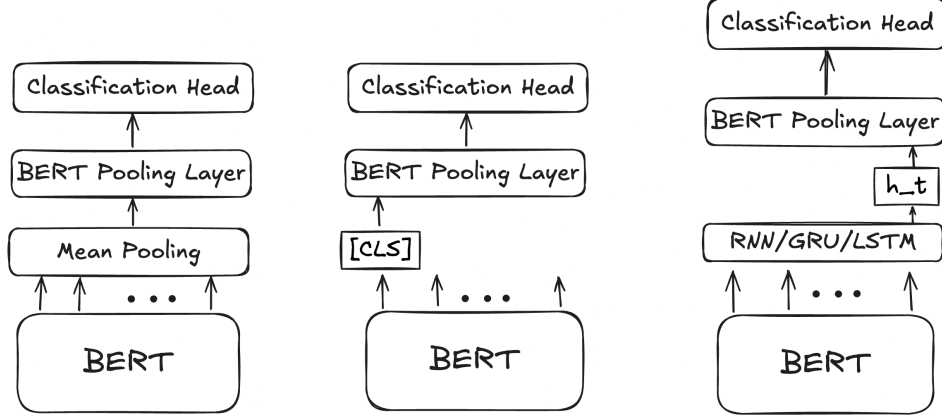


Figure 2: Words (tokens) Pooling strategy. The left hand side is the mean pooling and in the middle, is the [CLS] pooling and the right hand side is RNN pooling. Note that [CLS] is the embeddings of [CLS] token and h_t denotes the last hidden state of the RNN type nn. All of them are the primitive of sentence embeddings.

Besides, no matter to freeze MLM’s weights or to go for fine-tuning, we still need to configure how to construct the sentence-level embeddings from token-level ones. Conventionally, there are three approaches: mean, [CLS] and RNN pooling. To have a general idea about the differences between these three strategies, please see Figure 2. Mean pooling is basically averaging over all token embeddings from the last layer. This pooling is adopted by SBERT [3] which shows that the mean pooling is better when constructing semantically meaningful sentence embeddings trained on siamese and triplet networks. Regarding [CLS] pooling, we rely on a special token, [CLS], which is the first token of all BERT models. For a simple baseline method, “bert-base-uncased” pre-trained model is chosen and its pooling layer (BERT Pooling Layer in Figure 2) has been pre-trained to predict whether a pair of sentences is consecutive. For this pre-training scheme, there is a feed forward network with sigmoid activation, taking the [CLS] embedding from the last layer as input and the outputs aim to capture the sequence-level semantics. We call this feed forward network “BERT pooling layer”. As there isn’t any statements in [2] explaining the purpose of this layer or discussing its effectiveness, we design an experiment to see whether having a trainable BERT pooling layer will boost the performance, or if simply leveraging the mean pooling token or [CLS] token as the input of the classification head is a better configuration.

Following up, we are also curious about the effectiveness of the next sentence prediction pre-training scheme, so we have another setting which abandon both mean pooling and [CLS] tokens and directly access the token embeddings within BERT and utilize a RNN type of neural network to capture the sentence-level semantics. Between RNN type of neural networks and linear classification layer, we plug the pooling layer borrowed from the original BERT model and make it trainable to enhance and smooth the sentence embeddings. So far, we have discussed how to configure MLMs to perform text classification. Next, we will focus on investigating whether the capacity of MLMs will affect the performance or, more recent proposed pre-training MLMs, for example, RoBERTa [4], SBERT [3], DistilBERT [5] DeBERTa [6], DeBERTa-v3 [7] can assist to obtain higher classification quality.

4 Hawkish-Dovish Index

The classifier labels a single sentence, whereas the object we ultimately want is one number attached to a point in time. Let S_t collect the sentences the FED releases in period t , and let $\hat{y}_s \in \{\text{dovish}, \text{hawkish}, \text{neutral}\}$ be the label our classifier assigns to sentence s , taken as the arg max over the three predicted probabilities. Write $H_t = \{s \in S_t : \hat{y}_s = \text{hawkish}\}$ and $D_t = \{s \in S_t : \hat{y}_s = \text{dovish}\}$ for the hawkish and the dovish sentences of that period. The hawkish-dovish index is the net hawkish share of the period’s sentences:

$$\text{hdi}_t = \frac{|H_t| - |D_t|}{|S_t|} \quad (1)$$

Two properties follow from the definition alone. First, $\text{hdi}_t \in [-1, 1]$: each sentence carries exactly one label, so H_t and D_t are disjoint subsets of S_t and therefore $|H_t| + |D_t| \leq |S_t|$; the numerator is then bounded in absolute value by the denominator, and the index reaches 1 only when every sentence is hawkish and -1 only when every sentence is dovish. Second, neutral sentences never enter the numerator but do enter the denominator, so a period in which the FED speaks at length without committing to a direction is pulled toward zero rather than left undefined. This is the same aggregation [1] apply to their document-level measure, which lets the two series be compared directly once the classifier underneath is swapped.

5 Results

To find hyperparameters that have the best generalization capability, we split the original training data from [1] into training and validation set where validation set accounts for 20%. Then, we conduct hyperparameters tuning using validation set, and specifically, we refer to those which are highly related to neural network training or the architecture of a specific neural network, such as the batch size, learning rate and dimension of RNN’s hidden state, etc. The original test data is used to compare performance across different classifier design or different configurations before training such as the number of layers to use for extracting word embeddings. Basically, all metrics reported in following tables are evaluated on the test data.

For the evaluation metric, we decide to use the average precision which basically is the Riemann sum over the precision-recall curve. Let $\mathbf{y} \in \{0, 1, 2\}^n$ to be the class labels and 0, 1 and 2 means dovish, hawkish and neutral respectively. $\mathbf{P}_{n \times K}$ where $K = 3$ is the predicted probability. P_{ik} is the probability of the i -th sample being classified to the class $k - 1$. For each class k , we measure the area under precision-recall curve through a set of precision recall $\{(\text{precision}_{k,j}, \text{recall}_{k,j})\}_j$ calculated based on a monotonic sequence of threshold probability (ξ_j) where $\xi_j \in (0, 1)$. To be more detailed, to plot the precision-recall curve for each class k , the problem is transformed to a binary classification with labels denoted as whether belonging to class k or not and then calculating $\text{precision}_{k,j}$ and $\text{recall}_{k,j}$ using threshold ξ_j . Namely, within class, we consider one-vs-rest scenario and treat each class separately and independently. The average precision score of class k ap_k is defined as follows:

$$\text{ap}_k(\mathbf{y}, \mathbf{P}) = \sum_j \text{precision}_{k,(j+1)} (\text{recall}_{k,(j+1)} - \text{recall}_{k,j}) \quad (2)$$

Basically, for a low threshold ξ_j it’s more easy to classify a sample to be class k and thereby the recall for that class will be high whereas the precision will be low. The trade-off between precision and recall excel models that have superior capabilities to simultaneously maximize toward two directions.

5.1 Vanilla BERT Classifier

5.1.1 Tokens Embeddings Pooling Strategy

In this section, we aim at identifying the best pooling strategy and we start from various configurations of RNN pooling. There are a few options to adjust for RNN pooling. we first conduct experiment on which neural network architectures, RNN, GRU or LSTM to use for pooling word embeddings. We simply apply BERT to obtain word embeddings and specifically, we extract embeddings from the last layer of “bert-based-uncased” pre-trained model and then input them into various types of RNN architectures. The remained parts of neural networks are demonstrated in Figure 2.

Table 1: Results of utilizing different types of RNN architectures for RNN pooling strategy.

NN	d. ap	h. ap	n. ap	num epochs
RNN	0.3459	0.4223	0.6960	75
GRU	0.3936	0.4299	0.7057	85
LSTM	0.4047	0.4385	0.7307	55

In Table 1, we can observe the difference in performance between three types of architectures is small, especially between GRU and LSTM, so in the next stage of experiment, we jointly test on these two with different number of layers used for extracting word embeddings from BERT. Note that hyperparameters to reproduce Table 1 can be found in Table 6.

Word embeddings are generated based on sub-word token embeddings within BERT and configurations include how to transform sub-word embeddings to word embeddings, number of layers involved for embeddings construction and pooling strategy to aggregate output embeddings across layers. Typically, word embeddings are the first associated sub-words’ embeddings. Other options are to choose last sub-word instead of the first or to average over all sub-words in the corresponding words. However, we think this is a minor factor as how we stack neural networks and which pooling approaches seems to naturally be more crucial to obtain good classification results. Therefore, we decide to remain the default setting. Then, we try out different number of layers to generate embeddings and if the number of layers is set to be greater than one, then average pooling is adopted rather than concatenating embeddings across layers.

Table 2: Exploration on number of layers set to generate word embeddings as the input of neural network.

config	d. ap	h. ap	n. ap	num epochs
LSTM, last layers	0.4047	0.4385	0.7307	55
LSTM, all layers	0.3991	0.4538	0.7380	75
GRU, last layers	0.3936	0.4299	0.7057	85
GRU, all layers	0.3580	0.4258	0.6611	83

From Table 2, we can see that the best performant configurations will be LSTM with the input embeddings that generated by averaging over all layers of word embeddings. However, this approach has more parameters to train compared to the other types of pooling strategies which only have a linear classification head or potentially additional one layer of feed forward network. Though being expensive, the performance is actually not competitive with the others, as we will see in the following experiments.

When considering the mean and pooling strategy, we start to puzzle whether we should train weights of BERT pooling layer that adapt to our task. For meaning pooling, we decide whether to place the BERT pooling layer before the classification head while for [CLS] pooling we decide whether to take the last layer [CLS] embeddings or the pooler_output as the input of classification head. pooler_output is the second return of the “bert-base-uncased” Hugging Face model. For more information about pooler_output, please see: https://huggingface.co/docs/transformers/v4.46.3/en/model_doc/bert#transformers.BertModel.forward. Besides, if we select to use the [CLS] embeddings, we can also decide whether to place the BERT pooling layer before the classification head.

Table 3: [CLS] v.s. last layer mean pooling as well as justifying the need of BERT Pooling Layer. BPL is the abbreviation of “BERT pooling layer” and we are using bert-base-uncased in this table.

config	d. ap	h. ap	n. ap	num epochs
[CLS] w/o BPL	0.4158	0.4151	0.7797	129
[CLS] w/ BPL	0.3786	0.3994	0.7272	70
pooler_output (naturally w/o BPL)	0.3896	0.3873	0.7493	5000
last layer mean w/o BPL	0.5012	0.4844	0.7887	280
last layer mean w/ BPL	0.4766	0.4620	0.7756	195

Table 3 demonstrates that simply utilizing the [CLS] or average tokens embeddings are better. Moreover, when including the BERT pooling layer, the dropout rate tends to be high, approximately 0.6, which might indicate the potential over-fitting issue.

5.1.2 Larger Model Capacities and More Advanced MLMs

Although in Table 3, we can see that mean pooling is much superior than [CLS] pooling, we suspect this phenomenon is specific to the bert-base-uncased model. Consequently, we include more advanced and larger models to have a detailed comparison between these two pooling strategies.

Table 4: Detailed comparison between mean and [CLS] pooling on wide range of pre-trained MLMs. The baseline method is the third entry of Table 3.

BERT type	pooling strategy	d. ap	h. ap	n. ap	num epochs
bert-large-uncased [2]	[CLS]	0.5200	0.4874	0.8283	410
	mean	0.5296	0.5606	0.8429	1075
roberta-base [4]	[CLS]	0.5246	0.4752	0.8469	1160
	mean	0.5111	0.5009	0.8214	665
deberta-base [6]	[CLS]	0.3418	0.3701	0.7582	1090
	mean	0.5313	0.5420	0.8408	140
deberta-v3-base [7]	[CLS]	0.4383	0.3914	0.7584	680
	mean	0.5268	0.4865	0.8221	135
distilbert-base-uncased [5]	[CLS]	0.4548	0.4550	0.8065	234
	mean	0.5292	0.5107	0.7913	215
distilroberta-base [5]	[CLS]	0.5408	0.5459	0.8331	2870
	mean	0.5317	0.5378	0.8226	885

Table 4 does not settle on one pooling strategy. Mean pooling wins by a wide margin on the two DeBERTa checkpoints, 0.5313 against 0.3418 on the dovish class for deberta-base, and by a narrower one on bert-large-uncased and distilbert-base-uncased, whereas [CLS] pooling wins on distilroberta-base across all three classes and on roberta-base for the dovish and the neutral class. The gap between the two strategies is therefore model-specific, and it is often wider than the gap between two models under a fixed strategy, so pooling has to be picked per checkpoint rather than once for the whole study. We carry distilroberta-base with [CLS] pooling into Table 5 as the baseline, since it takes the best dovish-class score of Table 4 and is among the cheapest models near the top.

Table 5: Detailed comparison among various fine-tuned MLMs

	pooling strategy	d. ap	h. ap	n. ap	num epochs
(baseline) distilroberta-base [5]	[CLS]	0.5408	0.5459	0.8331	2870
fine-tuned on text classification task					
distilbert-base-uncased-finetuned-sst-2-english	mean	0.5292	0.4685	0.7955	540
twitter-roberta-base-sentiment-latest [8]	[CLS]	0.6272	0.5514	0.8459	1019
roberta-hate-speech-dynabench-r4-target [9]	[CLS]	0.4632	0.4298	0.7890	2850
roberta-base-go_emotions	[CLS]	0.5246	0.5034	0.8262	605
fine-tuned on sentence similarity task					
all-mpnet-base-v2 [3]	-	0.5988	0.5880	0.8316	190
multi-qa-mpnet-base-dot-v1	-	0.6162	0.5750	0.8378	360
all-MiniLM-L12-v2	-	0.5479	0.5480	0.8257	380
all-MiniLM-L6-v2	-	0.5670	0.5474	0.8186	555
finance-embeddings-investopedia	-	0.6170	0.4943	0.8156	2210
fine-tuned on economic, financial related data					
CentralBankRoBERTa-sentiment-classifier [10]	[CLS]	0.5406	0.5801	0.8196	365
distilroberta-finetuned-financial-news-sentiment-analysis	[CLS]	0.5201	0.5673	0.8115	435
financial-roberta-large-sentiment	[CLS]	0.6090	0.5477	0.8142	704
deberta-v3-ft-financial-news-sentiment-analysis	mean	0.5509	0.5277	0.8145	634
distilroberta-finetuned-financial-text-classification	[CLS]	0.5314	0.5182	0.8349	2789

Table 5 groups the fine-tuned checkpoints by the task each one was adapted to, and the pattern is that adaptation buys accuracy on the two classes the index depends on but not on the neutral class. Every group contains at least one checkpoint above the baseline’s 0.5408 on the dovish class, with twitter-roberta-base-sentiment-latest the best of all at 0.6272, and all-mpnet-base-v2 lifts the hawkish class to 0.5880 against the baseline’s 0.5459. On the neutral class, by contrast, nothing in Table 5 beats the 0.8469 that plain roberta-base already reaches in Table 4. Two further points are worth noting. The sentence-similarity checkpoints need no pooling decision from us at all, since they ship as sentence encoders whose pooling is fixed by the sentence-transformers recipe, which is why their pooling column reads “-”, and most of them converge within a few hundred epochs against the thousands that several [CLS]-pooled encoders need. Adaptation to financial text alone is also not sufficient: CentralBankRoBERTa, the checkpoint closest to our own domain, lands at 0.5406, 0.5801 and 0.8196, better than the baseline on the hawkish class but not on the dovish or the neutral one.

6 Conclusion

We set out to build a hawkish-dovish index, and because such an index is an average of sentence labels, the paper is spent on the classifier that produces them. Three results come out of the experiments. First, RNN pooling is the wrong place to spend parameters: the LSTM variants of Table 1 and Table 2 train the largest head of the three strategies and still lose to a linear head on mean-pooled embeddings in Table 3. Second, the BERT pooling layer, inherited from the next sentence prediction objective, hurts under both mean and [CLS] pooling, and the dropout rate selected on the validation set climbs to roughly 0.6 whenever it is present, which reads as over-fitting rather than as useful extra capacity. Third, what the encoder was pre-trained or fine-tuned on matters more than either its size or its pooling strategy: the strongest checkpoints adapted to sentence similarity and to financial text in Table 5 beat the best generic encoder of Table 4 on the dovish and the hawkish class, which are exactly the two classes entering the numerator of the index.

Three limitations bound these numbers. The weights of every masked language model reported here are frozen, so the comparison speaks to the quality of off-the-shelf representations rather than to what each model reaches once fine-tuned. Meta-learning and semi-supervised training, both of which target information beyond the semantics of a single sentence, are left to future work. And the index of Section 4 is defined but not yet estimated: computing the series with our best classifier and testing it against interest rate and equity moves around FOMC releases is the natural next step.

References

- [1] Agam Shah, Suvan Paturi, and Sudheer Chava, “Trillion Dollar Words: A New Financial Dataset, Task & Market Analysis,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 6664–6679.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.
- [3] Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [4] Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach.” arXiv, 2019. doi: 10.48550/arXiv.1907.11692.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter.” arXiv, 2020. doi: 10.48550/arXiv.1910.01108.
- [6] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: DECODING-ENHANCED BERT WITH DISENTANGLED ATTENTION,” in *International Conference on Learning Representations*, 2021.
- [7] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa Using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [8] J. Camacho-collados *et al.*, “TweetNLP: Cutting-Edge Natural Language Processing for Social Media,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2022, pp. 38–49.
- [9] B. Vidgen, T. Thrush, Z. Waseem, and D. Kiela, “Learning from the Worst: Dynamically Generated Datasets to Improve Online Hate Detection,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 1667–1682.
- [10] M. Pfeifer and V. P. Marohl, “CentralBankRoBERTa: A Fine-Tuned Large Language Model for Central Bank Communications,” *The Journal of Finance and Data Science*, vol. 9, p. 100114, 2023.

A Hyperparameters

Table 6: Hyperparameters for reproducing Table 1

NN	batch size	ff dropout	hidden size	mlearning rate
RNN	712	0.17831726122503855	1630	0.000005124067949679128
GRU	951	0.3651567026673735	839	0.00022486434146088324
LSTM	914	0.5247106064278061	296	0.000379196455082216

Table 7: Hyperparameters for reproducing Table 2. In this experiment, to accelerate the hyperparameters tuning procedure, we don’t include learning reate and set it to be 0.0003. Moreover, the “last year” configurations are results from table 2 so please check out Table 6.

NN	batch size	ff dropout	hidden size
GRU, all layers	856	0.5307182093173641	1510
LSTM, all layers	589	0.6753626506605535	1072

Table 8: Hyperparameters for reproducing Table 3. Note that if a pooling strategy forward pass without BERT pooling layer, then the hyperparameter, ff_droup is missing due to the absence of the feed forward layer.

config	batch size	learning rate	ff dropout
[CLS] w/o BPL	1249	0.0016125052446018724	-
[CLS] w/ BPL	1376	0.0028038761877692085	0.21807975814570435
pooler_output (w/o BPL)	1262	0.001899782640557171	-
last layer mean w/o BPL	1257	0.0026519820779429453	-
last layer mean w/ BPL	1285	0.00006419921706975861	0.6681774351500311

Table 9: Hyperparameters for reproducing Table 4

BERT type	pooling strategy	batch size	learning rate
bert-large-uncased [2]	[CLS]	989	0.0008713511378426164
	mean	991	0.0006907953626803432
roberta-base [4]	[CLS]	273	0.002625125982467249
	mean	999	0.0020396514920047325
roberta-large [4]	[CLS]		
	mean	1217	0.002005058082419334
deberta-base [6]	[CLS]	277	0.0029992320113528164
	mean	825	0.0010413456844612199
deberta-v3-base [7]	[CLS]	276	0.0028182802485605986
	mean	434	0.001822410702887656
deberta-v3-large [7]	[CLS]		
	mean		
distilbert-base-uncased [5]	[CLS]	356	0.001789371487900345
	mean	1386	0.002619772623863048
distilroberta-base [5]	[CLS]	990	0.0027116716360027554
	mean	1067	0.00243682910729925

Table 10: Hyperparameters for reproducing Table 5

config	pooling strategy	batch size	learning rate
(baseline)			
fine-tuned on text classification task			
distilbert-base-uncased-finetuned-sst-2-english	mean	1263	0.0005367370559450229
twitter-roberta-base-sentiment-latest [8]	[CLS]	973	0.00026232503880942745
bert-base-personality	mean		
roberta-hate-speech-dynabench-r4-target [9]	[CLS]	521	0.002642845730327103
roberta-base-go_emotions	[CLS]	628	0.0005946350530922512
fine-tuned on sentence similarity task			
all-mpnet-base-v2 [3]	-	1214	0.002236325676472196
multi-qa-mpnet-base-dot-v1	-	1257	0.0008692165269508549
all-MiniLM-L12-v2	-	1140	0.002227794529004433
all-MiniLM-L6-v2	-	265	0.0011362651459171757
finance-embeddings-investopedia	-	603	0.00041280227631448094
fine-tuned on economic, financial related data			
CentralBankRoBERTa-sentiment-classifier [10]	[CLS]	1046	0.001577923661192927
distilroberta-finetuned-financial-news-sentiment-analysis	[CLS]	977	0.0006768239206391982
financial-roberta-large-sentiment	[CLS]	709	0.00027436125662059455
deberta-v3-ft-financial-news-sentiment-analysis	mean	926	0.0006603538511760396
distilroberta-finetuned-financial-text-classification	[CLS]	457	0.00005241291811691051